

Future-proof submdspan_mapping

Document #: P3663R3
Date: 2025-10-03
Project: Programming Language C++
LWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>
Tomasz Kamiński
<tomaszkam@gmail.com>

Contents

- 1 Author
- 2 Revision history
- 3 Abstract
- 4 Motivation
 - 4.1 Summary
 - 4.2 Four categories of slice types
 - 4.3 Why undefined behavior?
 - 4.4 Set of contiguous subrange slices was expanded once and may be again
 - 4.5 Other existing categories could be expanded
 - 4.6 WG21 may add a new slice category
- 5 Solution: Make submdspan canonicalize slices
 - 5.1 Summary
 - 5.2 Canonicalization rules
 - 5.3 How to apply canonicalization
 - 5.4 Canonicalization is reusable
 - 5.5 What about precondition checking?
 - 5.6 Why implementers should do this anyway
- 6 Performance
 - 6.1 Speculation on compile-time and run-time effects
 - 6.2 submdspan can skip canonicalization for Standard layout mappings

- 7 Implementation
 - 7.1 Configuration and build
 - 7.2 Benchmark
 - 7.2.1 Benchmark code details
 - 7.2.2 Preliminary results
 - 7.2.3 Preliminary conclusions
- 8 Proposed wording
 - 8.1 Increment `__cpp_lib_submdspan` feature test macro
 - 8.2 Change `[mdspan.syn]`
 - 8.3 Context from `[mdspan.extents.overview]` (NO CHANGES)
 - 8.4 Change `[mdspan.sub.overview]`
 - 8.5 Change `[mdspan.sub.helpers]`
 - 8.6 Add section `[mdspan.sub.canonical]`
 - 8.7 Change section `[mdspan.sub.extents]`
 - 8.8 Requirements of all `submdspan_mapping` customizations
 - 8.8.1 New section `[mdspan.sub.sliceable]`, “Sliceable layout mapping requirements”
 - 8.9 Change `[mdspan.sub.map.common]`
 - 8.10 Change `[mdspan.sub.map.left]`, `layout_left` specialization of `submdspan_mapping`
 - 8.11 Change `[mdspan.sub.map.right]`, `layout_right` specialization of `submdspan_mapping`
 - 8.12 Do not change `[mdspan.sub.map.stride]`, `layout_stride` specialization of `submdspan_mapping`
 - 8.13 Change `[mdspan.sub.map.leftpad]`, `layout_left_padded` specialization of `submdspan_mapping`
 - 8.14 Change `[mdspan.sub.map.rightpad]`, `layout_right_padded` specialization of `submdspan_mapping`
 - 8.15 Change `[mdspan.sub.sub]`, `submdspan` function template

§ 1 Author

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Tomasz Kamiński (tomaszkam@gmail.com)

§ 2 Revision history

- Revision 0 to be submitted 2025-03-17
 - 2025-04-15: Reviewed by LEWG and forwarded to LWG.
- Revision 1 to be submitted 2025-05-19 for the pre-Sofia mailing
 - Add Mandates and Preconditions to *canonical-ice* that the input be representable as a value of type `IndexType`.
 - Fix *subtract-ice* to use `constant_wrapper` arithmetic: that is, apply *canonical-ice* first, then subtract the resulting `constant_wrapper` objects.
 - Use structured binding for pair slices, not `get`; reorder Mandates and Returns clauses to put the structured binding part last. (For Mandates, this means that implementations need only try the structured binding; if it fails, it's ill-formed.)
 - `submdspan` can't actually be constrained on the specific `submdspan_mapping(input.mapping(), slices...)` call being well-formed, because there's no way to constrain on a structured binding declaration being well formed. (Structured binding is a declaration, while constraints require expressions.) Instead, change the constraint on `submdspan` to be that `submdspan_mapping(input.mapping(), full_extent, ..., full_extent)` is well-formed (call that an exposition-only concept *mapping-sliceable-with-full-extents*), and make it just a Mandate that `submdspan_mapping(input.mapping(), slices...)` be well-formed.
 - Add a new section for requirements on sliceable mappings, with a new named requirement “sliceable layout mapping,” and an exception for the standard mappings. The requirements are currently expressed in parts of `[mdspan.sub.map.common]` and `[mdspan.sub.sub]` (e.g., 5.2 and 5.3, with Note 2 after 5.3).
 - Add new function `submdspan_canonicalize_slices` to `<mdspan>` header synopsis.
 - Fix non-wording Abstract and the rest of the non-wording sections to say that we do actually propose these changes.
 - Add “Canonicalization is reusable” non-wording section to explain why `submdspan_canonicalize_mapping` is not exposition-only.
 - Update P2781 (`std::constant_wrapper`) references from R7 to R8.
 - Add an Editorial Note to explain that the standard layout mappings could accept all valid slice types, instead of just the canonical ones. This should alleviate

concerns about potential performance effects of canonicalization. However, retain (in Editorial Notes) suggested wording for the pass-through, just in case.

- Define an exposition-only function *check-static-bounds* to avoid repetition when stating Mandates on canonical and valid slice types.
- Revision 2 to be submitted 2025-07-15 for the post-Sofia mailing
 - Add new implementation link.
 - Based on implementation experience with *canonical-ice*,
 - specify its return value using *cw* instead of *constant_wrapper* (to avoid issues with the first template parameter of *constant_wrapper* actually being of an exposition-only type instead of the expected *IndexType*), and
 - cast the result of *index-cast* to *IndexType* before using it as the constant value in *cw*.
 - Based on implementation experience with *check-static-bounds*,
 - rewrite *check-static-bounds* wording to use only $S_k\{\}$ instead of s_k , so that all of the expressions in the wording are valid constant expressions;
 - specify *check-static-bounds* to take only one slice type as an explicit template parameter, instead of taking all the slices as a parameter pack (as taking the slices as function parameters would make calling *check-static-bounds* from a function taking slice parameter(s) not a constant expression); and
 - check wording of *check-static-bounds* so that it only assumes that types are default constructible in constant expressions if they are *integral-constant-like*.
 - Based on implementation experience with *submdspan*, in *submdspan*’s “*Effects: Equivalent to*” code,
 - fix formatting of *canonical_slices*,
 - add drive-by fix for *src.data_handle()* (original was *src.data()*), and
 - add drive-by fix adding missing *typename* before *AccessorPolicy::offset_policy*.
 - Based on LWG reflector discussion on 2025/06/02,
 - fix *first_* and *last_* wording to say e.g., “ s_{k0} produced by *auto* [s_{k0} , $_]$ $=s_k$,” instead of “ s_{k1} if the structured binding declaration *auto* [s_{k0} , s_{k1}] $=s_k$; is well-formed”;

- fix *check-static-bounds Returns* clause to follow the above pattern instance of saying “if the structured binding declaration `auto [s_k0, s_k1] = declval<Slice>();` is well-formed...”;
- adjust *check-static-bounds Returns* clause to make the function well-formed in cases where the input slice is one of the three valid non-pair-like slice types but would otherwise return unknown; and
- adjust *submdspan_canonicalize_slices Returns* clause so that the last (pair-like) case follows the same pattern as *first_* and *last_*.
- Ensure that *canonical-ice* returns either `cw<IndexType(Value)>` or `IndexType(Value)` for the value `Value`.
- Expand discussion of implementation, and add preliminary benchmark results.
- Revision 3 to be submitted 2025-10-03
 - Add Tomasz Kamiński to the author list
 - Major wording changes based on LWG review.
 - Wording is now more mathematical and less a description of an implementation (as R2 wording was).
 - Some exposition-only functions proposed in previous revisions (e.g., *canonical-ice*, *subtract-ice*, and *check-static-bounds*), as well as some preexisting exposition-only functions (e.g., *first_* and *last_*), have been renamed, replaced, or removed.
 - “Unit-stride slice” has been redefined to depend only on `IndexType` instead of a layout mapping, and to apply only to canonical slice types. Its definition has been moved up from `[mdspan.sub.map.common]` to `[mdspan.sub.slices]`.
 - Update non-wording sections to reflect wording changes

§ 3 Abstract

Currently, the `submdspan` function can call a `submdspan_mapping` customization with any valid slice type. This means that `submdspan_mapping` customizations may be ill-formed, possibly even without a diagnostic, if future C++ versions expand the set of valid slice types. This will happen if [P2769R3](#) is merged into the Working Draft for C++29, as that would generalize *tuple-like* from a concrete list of types to a concept. It may also happen in the future for other categories of slice types.

We propose fixing this with a “canonicalization” approach. First, define a fixed set of “canonical” slice types that represent all valid slices, and a function

`submdspan_canonicalize_slices` to map valid slices to their canonical versions. Then, change `submdspan` so it always canonicalizes its input slices, and so it only ever calls `submdspan_mapping` customizations with canonical slices.

§ 4 Motivation

§ 4.1 Summary

The current `submdspan` wording has the following constraint ([\[mdspan.sub.sub\] 3.2](#)).

... the expression `submdspan_mapping(src.mapping(), slices...)` is well-formed when treated as an unevaluated operand.

However, nothing currently requires `submdspan_mapping` to be ill-formed when any of its slices is *not* a valid slice type. Thus, the `submdspan` function can call a `submdspan_mapping` customization with any slice type that is valid for `submdspan`. This means that the following scenario invokes undefined behavior.

1. A user defines a layout mapping `my_layout::mapping` with a C++26 - compatible `submdspan_mapping` customization.
2. A later C++ version adds a new valid slice type `new_slice`.
3. `submdspan` is called with `my_layout::mapping` and a `new_slice` slice.

This is true whether the new slice type is in one of the existing four categories of slice types or is in a new category. It is a scenario that has already occurred and may occur again, as we will explain below.

§ 4.2 Four categories of slice types

The Working Draft has four categories of slice types. Let `S` be an input slice type of `submdspan`.

1. “Full”: `S` is convertible to `full_extent_t`. This means “all of the indices in that extent.”
2. “Collapsing”: `S` is convertible to (the layout mapping’s) `index_type`. This means “fix the slice to view that extent at only that index.” Each collapsing slice reduces (“collapses”) the result’s rank by one.
3. “Contiguous subrange”: `S` models *index-pair-like*`<index_type>`, or `S` is a `strided_slice` specialization whose `stride_type` is *integral-constant-like* with value 1. A contiguous subrange slice represents a contiguous range of indices in that extent. The fact that the indices are contiguous (in other words, that they have stride 1) is known at compile time as a function of the slice’s type alone. The special case of `strided_slice` with compile-time stride 1 is the only way to

represent a contiguous index range with run-time offset but compile-time extent. (See [P3355r2](#), which was voted into the Working Draft for C++26 at the 2024 Wrocław meeting.)

4. “Strided”: `S` is a specialization of `strided_slice`, but is not a contiguous subrange. It may represent a noncontiguous set of indices in that extent.

The current Working Draft uses the term *unit-stride slice* (defined in [P3355](#)) to include both full and contiguous subrange slices.

§ 4.3 Why undefined behavior?

Why would calling a C++26 - compatible `submdspan_mapping` customization with a post - C++26 slice type be undefined behavior? Why wouldn't it just be ill-formed? The problem is that the customization might actually be well-formed for new slice types, but it might do something that `submdspan` doesn't expect. For example, it might violate preconditions of the `submdspan_mapping` customization, or it might produce a `submdspan_mapping_result` that violates postconditions of `submdspan` (e.g., views the wrong elements). This could happen for two reasons.

1. The user has overloaded the customization incorrectly. It works correctly for all C++26 slice types, but incorrectly maps the new slice type to one of the four C++26 cases.
2. The user intends for the customization to be well-formed with invalid slice types. It might have behavior that makes sense for the user, but not for `submdspan`, as in the following example.

```
struct my_tag_slice_t {
    template<size_t k>
    friend constexpr size_t get() const {
        if constexpr (k == 0) {
            return 42;
        }
        else if constexpr (k == 1) {
            return 100;
        }
        else {
            static_assert(false);
        }
    }
};

namespace std {
    struct tuple_size<my_tag_slice_t>
        : integral_constant<size_t, 2> {};
    template<size_t I>
    struct tuple_element<I, my_tag_slice_t> {
        using type = size_t;
    };
} // namespace std
```

```

template<class... Slices>
friend constexpr auto
submdspan_mapping(const mapping& src, Slices... slices)
{
    if constexpr ((valid_cpp26_slice<Slices> && ...)) {
        // ... all slices are valid C++26 slices,
        // so do the normal C++26 thing ...
    }
    else if constexpr (is_same_v<Slices, my_tag_slice_t> && ...) {
        // my_tag_slice_t is not a valid C++26 slice,
        // but it has an unambiguous interpretation
        // as a pair of indices.
        return std::string("Hello, I am doing something weird");
    }
    else {
        static_assert(false);
    }
}

```

§ 4.4 Set of contiguous subrange slices was expanded once and may be again

WG21 has already expanded the set of contiguous subrange slice types once, and is likely to do so again. This is because it depends on the exposition-only concept *pair-like*. Adoption of [P2819R2](#) into the Working Draft for C++26 at the 2024 Tokyo meeting made complex a *pair-like* type, and therefore a valid slice type for `submdspan`. The *pair-like* concept depends in turn on the exposition-only concept *tuple-like*. [P2769R3](#), currently in LEWG review, proposes generalizing *tuple-like* from a concrete list of types to a concept. That would make user-defined pair types valid slice types.

Before adoption of [P2819R2](#), a user-defined layout mapping's `submdspan_mapping` customization could determine whether a type is *T* is *pair-like* by testing exhaustively whether it is `array<X, 2>` for some *X*, `tuple<X, Y>` or `pair<X, Y>` for some *X* and *Y*, or `ranges::subrange` with `tuple_size` 2. P2819 would break a customization that uses this approach. Similarly, adoption of [P2769R3](#) would break customizations that add `complex<R>` (for some *R*) to this list of types to test exhaustively.

It would be reasonable for `submdspan` to generalize *pair-like* into any type for which structured binding into two elements is well-formed. If the two elements are both convertible to `index_type`, then this has an unambiguous interpretation as a contiguous subrange. Thus, users should write their customizations generically to that interface. However, in terms of the Standard, there are two issues with that approach.

1. Standard Library implementers need to assume that users read the Standard as narrowly as possible. The current C++ Working Draft does not forbid calling a `submdspan_mapping` customization with a user-defined pair type, but it also does not restrict its behavior in that case.

2. This approach would not help if other slice categories are expanded, or if a new slice category is added, as the following sections explain.

§ 4.5 Other existing categories could be expanded

The only types currently in the “strided” category are specializations of `strided_slice`. Future versions of C++ might expand this category. In our view, this would have little benefit. Nevertheless, nothing stops WG21 from doing so. For example, C++29 might (hypothetically) make a strided slice any type that meets the following exposition-only *strided-slice*<index_type> concept.

```
template<class T>
concept integral_not_bool = std::is_integral_v<T> &&
    ! std::is_same_v<T, bool>;

template<class IndexType, class T>
concept strided-slice = semiregular<T> &&
    convertible_to<T::offset_type, IndexType> &&
    convertible_to<T::extent_type, IndexType> &&
    convertible_to<T::stride_type, IndexType> &&
    requires(T t) {
        { t.offset } -> convertible_to<IndexType>;
        { t.extent } -> convertible_to<IndexType>;
        { t.stride } -> convertible_to<IndexType>;
    };
```

The following user-defined type `my_slice` would meet *strided-slice*<size_t>, even though

- it is not convertible to or from `strided_slice<size_t, size_t, size_t>`;
- it has a base class, which `strided_slice` specializations cannot, per [mdspan.sub.strided.slice] 2; and
- it has members other than `offset`, `extent`, and `stride`, which `strided_slice` cannot, per [mdspan.sub.strided.slice] 2.

```
struct my_base {};
```

```
struct my_slice : public my_base {
    using offset_type = size_t;
    using extent_type = size_t;
    using stride_type = size_t;

    offset_type offset;
    extent_type extent;
    stride_type stride;

    // User-declared constructor makes this not an aggregate
    my_slice(extent_type ext) : offset(0), extent(ext), stride(1) {}

    // strided_slice is not allowed to have extra members
```

```

std::string label = "my favorite slice label";
std::vector<int, 3> member_function() const { return {1, 2, 3}; }

// my_slice is not convertible to or from strided_slice
template<class Offset, class Extent, class Stride>
my_slice(strided_slice<Offset, Extent, Stride>) = delete;

template<class Offset, class Extent, class Stride>
operator strided_slice<Offset, Extent, Stride>() const = delete;
};

```

Now suppose that a user has defined a `submdspan_mapping` customization in a C++26-compatible way for their custom layout mapping `custom_layout::mapping`. Then, `my_slice` likely would not work for this layout mapping in C++29, even though the hypothetical `_strided_slice_` concept makes it possible to interpret `my_slice` unambiguously as a strided slice, and to write generic code that does so.

Unlike with contiguous subrange slices, users have no way to anticipate all possible ways that future Standard versions might expand the set of valid strided slice types. For example, C++29 could decide that strided slices must be aggregate types (which would exclude the above `my_base`), or that any type for which a structured binding into three elements is well-formed is a valid strided slice. Furthermore, `my_slice` with `stride = cw<1, size_t>` should reasonably count as a contiguous subrange slice type; expanding one set risks conflicts between sets.

§ 4.6 WG21 may add a new slice category

The same problem would arise if future versions of C++ introduce a new category of slices. For example, suppose that C++29 hypothetically adds an “indirect slice” category that takes an arbitrary tuple of indices to include in that extent of the slice. The Mandates of `submdspan` ([`mdspan.sub.sub`] 4.3) make it ill-formed to call with a new kind of slice, so this hypothetical C++29 addition would need to add a new clause there. That would not be a breaking change in itself. The problem is that nothing currently requires `submdspan_mapping` to be ill-formed when any of its slices is not a valid slice type.

§ 5 Solution: Make `submdspan` canonicalize slices

§ 5.1 Summary

There is a tension between `submdspan` users and custom mapping authors. Users of `submdspan` want it to work with all kinds of slice types. For example, they might want to write their own tuple types that are standard-layout if all their template parameters are (unlike `std::tuple`), and would expect `submdspan` to work with them. In contrast, users who customize `submdspan_mapping` for their own layout mappings want a “future-proof”

interface. They only want `submdspan` to call `submdspan_mapping` for a known and unchanging set of types.

We propose resolving this tension by changing `submdspan` so that it “canonicalizes” its slice inputs before passing them to `submdspan_mapping`. The `submdspan` function still takes any valid slice types, but it maps each input slice type to a “canonical” slice type according to rules that we will explain below. As a result, a `submdspan_mapping` customization would ever be called with the following concrete list of slice types.

1. `full_extent_t`;
2. `index_type` (that is, the layout mapping’s `index_type`);
3. `constant_wrapper<Value, index_type>` for some `Value`; or
4. `strided_slice` where each member is either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`.

The `constant_wrapper` class template comes from [P2781](#). LEWG design-approved P2781R7 (with design changes not relevant to this proposal, which are expressed as P2781R8) and forwarded it to LWG on 2025/03/11 for C++26. The only aspect of a type that can vary in the above list is `constant_wrapper`’s `Value`.

This solution would make it always ill-formed to call an existing `submdspan_mapping` customization with a new kind of slice. With the status quo, calling an existing `submdspan_mapping` customization with a new kind of slice would be ill-formed *at best*, and could be a precondition violation at worst, because nothing prevents `submdspan` from ever calling existing `submdspan_mapping` customizations with new slice types.

§ 5.2 Canonicalization rules

Here are the canonicalization rules for an input slice `s` of type `S` and a input mapping whose extents type has index type `index_type`. The result depends only on the input mapping’s extents type and the slices.

1. If `S` is convertible to `full_extent_t`, then `full_extent_t`;
2. else, if `S` is convertible to `index_type`, then `canonical-index<index_type>(s)`;
3. else, if `S` is a specialization of `strided_slice`, then

```
strided_slice{
    .offset = canonical-index<index_type>(s.offset),
    .extent = canonical-index<index_type>(s.extent),
    .stride = canonical-index<index_type>(s.stride)
}
```

4. else, if the structured binding declaration

```
auto [first, last] = std::move(s);
```

is valid and if `first` and `last` are both convertible to `index_type`, then

```
strided_slice{
    .offset = canonical-index<index_type>(first),
    .extent = canonical-index<IndexType>(c_last - c_first),
    .stride = cw<index_type(1)>
}
```

The exposition-only function *canonical-index* preserves “compile-time-ness” of any *integral-constant-like* arguments, and forces all inputs to either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`. The function also has Mandates or Preconditions (depending on whether its argument is *integral-constant-like*) that its argument is representable as a value of type `index_type`. This lets implementations check for overflow. The `cw` variable template comes from [P2781](#).

§ 5.3 How to apply canonicalization

The function `submdspan_canonicalize_slices` canonicalizes all the slices at once. It takes as arguments the input mapping’s extents and the input slices of `submdspan_mapping`. Canonicalization makes `submdspan` equivalent to the following code.

```
auto [...canonical_slices] =
    submdspan_canonicalize_slices(src.extents(), slices...);
auto sub_map_result =
    submdspan_mapping(src.mapping(), canonical_slices...);
return mdspan(
    src.accessor().offset(src.data_handle(), sub_map_result.offset),
    sub_map_result.mapping,
    typename AccessorPolicy::offset_policy(src.accessor()));
```

The statement starting with `auto [...canonical_slices]` above uses the “structured bindings can introduce a pack” feature introduced by [P1061R10](#) into C++26. The `submdspan_canonicalize_slices` function takes the extents for two reasons.

1. A slice’s canonical type depends on the extents’ `index_type`.
2. Knowing the extents lets it enforce preconditions on the slices, so that the resulting canonical slices don’t need to be checked again.

§ 5.4 Canonicalization is reusable

We expose `submdspan_canonicalize_slices`, instead of making it exposition-only, because users may want to use it to build their own functionality analogous to `submdspan`. For example, consider a C++ type `small_matrix<T, M, N>` representing a dense rank-2 array with element type `T` and compile-time extents `M` and `N`. (This could be a library exposure of a compiler extension, such as [Clang’s matrix type extension](#).) This type would expose access to individual elements, but would *not* expose subviews. That is, getting an `mdspan` that views any subset of elements would be forbidden. However, `small_matrix`

could expose *copies* of subsets of elements, via a function `submatrix` that takes the same slice specifiers as `submdspan`.

```
small_matrix<float, 5, 7> A = /* ... */;
auto A_sub = submatrix(A,
    strided_slice{.offset=1, .extent=4, .stride=3}, // 1, 4
    strided_slice{.offset=0, .extent=7, .stride=2}); // 0, 2, 4, 6
```

If `A` were a `layout_right` `mdspan`, then result of `submdspan` with those slices would be `layout_stride`. However, `A` is not an `mdspan`. Its layout might be hidden from users. Even if `small_matrix` layouts are known, there's no need for `submatrix` to return layouts in the same way as `submdspan` does. For example, it might “pack” the result into a contiguous `layout_right` `small_matrix`. Nevertheless, the slice specifiers have the same interpretation for `submatrix` as for `submdspan`; they are independent of the input's and result's layout mappings. Thus, users who want to implement `submatrix` could use `submdspan_canonicalize_slices` to canonicalize `submatrix`'s slice arguments.

This example explains why `submdspan_mapping` has a name with the word “`submdspan`” in it, even though it is purely a function of layout mappings and need not be tied to `mdspan`. Calling `submdspan_mapping(mapping, slices...)` returns the mapping that `submdspan(input, slices...)` would have for `mapping = input.mapping()`.

§ 5.5 What about precondition checking?

The `submdspan` function imposes preconditions on both its slice arguments and on the result of `submdspan_mapping` ([[mdspan.sub.sub](#)] 5). The preconditions on `submdspan_mapping`'s result require that the user's customization give a mapping that views exactly those elements of the input `mdspan` that are specified by the slices. That is, the customization has to be correct, given valid input slices. Enforcement of this is the user's responsibility.

Enforcement of preconditions on the input slices is the implementation's responsibility. This depends only on the input slices and the input mapping's extents. Just as `mdspan` can check bounds for the input of the `at` function using the input and its extents, `submdspan` can enforce all preconditions on the slices using the input mapping's extents. The current wording expresses these preconditions identically in two places:

1. on the slice arguments of `submdspan_extents` ([[mdspan.sub.extents](#)] 3), and
2. on the slice arguments of `submdspan` ([[mdspan.sub.sub](#)] 5).

Passing the slices through `submdspan_extents` is not enough to ensure that `submdspan_mapping`'s slice arguments do not violate its (unstated) preconditions. This is because deciding the result mapping's type and constructing it may depend on the slices, not just the result's extents. Another way to say this is that different `slices...` arguments might result in the same extents, but a different layout mapping. For example, if `src` is a rank-1 `layout_left` mapping with extents `dims<1>{10}`, both `full_extent` and `strided_slice{.offset=0, .extent=src.extent(0), .stride=1}` would result in

extents<1>{10}, but different layouts (`layout_left` and `layout_stride`, respectively). This means that slice canonicalization is also an opportunity to avoid duplication of precondition-checking code.

There are two kinds of preconditions on slices:

1. that all indices represented by the slice can be represented as values of type `extents::index_type`, that is, that casting them to `extents::index_type` does not cause integer overflow; and
2. that all indices represented by slice `k` are in `[0, src.extent(k) )`.

The current wording expresses both preconditions via the exposition-only functions `first_` and `last_`. The specification of each of these uses `extents::index-cast` to preprocess their return values (see [\[mdspan.sub.helpers 4\]](#) and [\[mdspan.sub.helpers 9\]](#)). The `index-cast` exposition-only function [\[mdspan.extents.expo\] 9](#) is a hook to enforce the “no overflow” precondition. The “in the extent’s range” precondition is currently up to `submdspan` implementations to check.

Slice canonicalization needs to have at least the “no overflow” precondition, because any integer results are either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`. That is, it always casts input to `index_type`. We propose applying the “in the extent’s range” precondition to slice canonicalization as well. This would let `submdspan` implementations put all their precondition checking (if any) in the canonicalization function. Users who implement a `submdspan_mapping` customization could thus assume that the input slices are all correct.

§ 5.6 Why implementers should do this anyway

Implementers should canonicalize slices anyway, even if this proposal is not adopted. Suppose, for example, that P2769 (expanding the definition of *tuple-like*) is adopted into some future C++ version. Implementations might thus expose users to an intermediate period in which they implement C++26 but not C++29. During this period, users might write `submdspan_mapping` customizations to the narrower definition of *index-pair-like* (which depends on *tuple-like*). For example, users might write `std::get` explicitly, which would only work with Standard Library types that have overloaded `std::get`, instead of using structured binding to get the two elements. This would, in turn, encourage implementers of `submdspan` to “canonicalize” *index-pair-like* slice arguments (e.g., into `pair` or `tuple`) before calling `submdspan_mapping` customizations, as that would maximize backwards compatibility.

In summary, the possibility of later generalization of *tuple-like* would naturally lead high-quality implementations to the desired outcome. That would minimize code changes both for users, and for implementations (as the Standard Library currently has five `submdspan_mapping` customizations of its own). This proposal merely anticipates the expected implementation approach.

§ 6 Performance

§ 6.1 Speculation on compile-time and run-time effects

We have not measured the performance effects of this approach. Everything we write here is speculation.

In terms of compile-time performance, canonicalization would minimize the set of instantiations of both `submdspan_mapping` and `submdspan_extents` (which customizations of `submdspan_mapping` may call). Specification of `submdspan_canonicalize_slices` in terms of an exposition-only function template *canonical-slice*, whose template parameters are just `IndexType` and one slice type (rather than the whole pack of slice types), gives implementations the opportunity to minimize instantiations.

In terms of run-time performance, customizations would likely need to copy slices into their canonical forms. This would introduce new local variables. Slices tend to be simple, and are made of combinations of zero to three integers and empty types (like `constant_wrapper` or `full_extent_t`). Thus, it should be easy for compilers to optimize away all the extra types. On the other hand, if they can't, then the compiler may need to reserve extra hardware resources (like registers and stack space) for the extra local variables. This *may* affect performance, especially in tight loops.

If there are performance issues, users may mitigate them somewhat by preferring use of `strided_slice` over arbitrary “pair-like” types. It would be natural for users to define a function to create “pair-like” slices that returns `strided_slice` with `cw<1>` as the stride.

```
template<class BeginType, class EndType>
constexpr auto
make_index_pair(BeginType beg, EndType end) {
    return std::strided_slice{
        beg, end - beg, std::cw<1>
    };
}
```

§ 6.2 `submdspan` can skip canonicalization for Standard layout mappings

The proposed set of canonicalization rules would require turning every “pair-like” type into a `strided_slice`. Users of `mdspan`-like libraries such as [Kokkos](#) rely heavily on pair-like slices. As a result, implementations may want to optimize specially for them. By the as-if rule, `submdspan` for Standard layout mappings has permission to pass any `submdspan` slice type straight through to the layout mapping without canonicalization. This would permit direct use of “pair-like” types in a structured binding declaration, without intermediate conversion to `strided_slice`.

Note that this proposal restricts *all* submdspan_mapping customizations, including those for Standard layout mappings, only to accept canonical slice types. Thus, submdspan’s pass-through would need to use a nonpublic interface in the implementation of Standard layout mappings. The implementation of submdspan would not have a way to pass through noncanonical slice types for user-defined layout mappings.

§ 7 Implementation

[Pull Request 408](#) in the mdspan reference implementation’s [GitHub repository](#) implements this proposal.

§ 7.1 Configuration and build

The same branch can be used for testing with P3663 support either enabled or disabled. To enable P3663 support, set the CMake option MDSPAN_ENABLE_P3663 to ON.

The branch includes two different implementations of P3663 support. The default version requires C++20. The nondefault version requires the C++23 feature “deducing this,” and optionally can make use of the C++26 feature “[pack indexing](#)” (P2662R3) if available. The current development version of Clang (21.0.0) implements pack indexing; the latest release of GCC (15.1.0) does not. Thus, the code checks GCC version macros to decide whether to use the feature.

The default implementation has been tested with Clang 14 and GCC 11.4.0. The nondefault version has been tested with Clang 21.0.0 and GCC 15.1.0. To enable the default version, please use the following CMake options.

```
-DMDSPAN_ENABLE_P3663=ON \  
-DCMAKE_CXX_FLAGS="-std=c++20" \  
-DMDSPAN_CXX_STANDARD=20 \  
-DMDSPAN_CONSTANT_WRAPPER_WORKAROUND=ON
```

To enable the nondefault version, please use the following CMake options.

```
-DMDSPAN_ENABLE_P3663=ON \  
-DCMAKE_CXX_FLAGS="-std=c++2c" \  
-DMDSPAN_CXX_STANDARD=26 \  
-DMDSPAN_CONSTANT_WRAPPER_WORKAROUND=OFF
```

§ 7.2 Benchmark

§ 7.2.1 Benchmark code details

The P3663 branch includes benchmarks in the source file benchmarks/submdspan/submdspan.cpp. We show results for “host_benchmark2” below. This iterates many times over all the elements of a rank-6 mdspan of uint8_t, and multiplies each one in place by 3u. Knowing the initial elements (which are generated randomly) and the number of iterations lets us compute the expected results

deterministically by repeated squaring modulo 256. Iteration over all the elements happens by recursive repeated slicing `submdspan(x, k, full_extent, ..., full_extent)` (where `full_extent, ..., full_extent` is not the actual syntax, but represents `rank() - 1` instances of `full_extent`). Slicing continues until the benchmark reaches a rank-1 `mdspan`, at which point the code just iterates over all the elements. Since the `mdspan` is `layout_right`, this iteration happens over contiguous data. We choose extents 4, 4, 4, 4, 4, 2 so that as much slicing as possible happens, while still keeping the working set in cache.

All slice arguments of `submdspan` are obfuscated in order to exercise the cost of slice conversions. The loop index `k` here is the `mdspan`'s `index_type`, but obfuscated by being stored in an `index_holder` wrapper type that is convertible to `index_type`. Likewise, `full_extent` in the expression above is actually `full_extent_wrapper_t{}`, an instance of an empty type that is convertible to `full_extent_t`.

The benchmarks shown here use the CMake flags and build configurations described above. Results generally use release builds (`CMAKE_BUILD_TYPE=Release`) without other CMake options that would change flags.

We show results for four different extents specializations.

1. `extents<int, 4, 4, 4, 4, 4, 2>` (rank 6, `IndexType=int`, all static extents)
2. `dims<6, int>{4, 4, 4, 4, 4, 2}` (rank 6, `IndexType=int`, all dynamic extents)
3. `extents<size_t, 4, 4, 4, 4, 4, 2>` (rank 6, `IndexType=size_t` (unsigned long), all static extents)
4. `dims<6, size_t>{4, 4, 4, 4, 4, 2}` (rank 6, `IndexType=size_t` (unsigned long), all dynamic extents)

The benchmark repeats iteration over all the elements of the `mdspan` 10^4 (10,000) times before a time measurement. We call this the “inner iteration count.” Google Benchmark then repeats this for a number of iterations that it decides – which we call the “outer iteration count” – before reporting a mean time over all outer iterations.

§ 7.2.2 Preliminary results

We show results with four different compilers.

1. Clang 21.0.0
2. GCC 15.1.0
3. Clang 14.0.0
4. GCC 11.4.0

For the first two compilers, we use the default version of the implementation, that assumes at least C++23. For the last two compilers, we use the nondefault “back-port” version that

assumes only C++20. We include the back-port version because it would be how many users – including users of Kokkos, who rely on the reference mdspan implementation as the back-end for Kokkos::View – would experience P3663’s changes.

The following results were collected on an 11th Gen Intel(R) Core(TM) i7-11850H running at 2496 MHz. It reports 16 processors, with L1 and L2 caches shared between pairs of processors and the L3 cache shared over all processors. The L1 data cache is 48 KiB, the L1 instruction cache is 32 KiB, the L2 unified cache is 1280 KiB, and the L3 unified cache is 24576 KiB.

Benchmarks were run on Windows Subsystem Linux 2 (Ubuntu 22 LTS) under Windows 11. We show the “CPU” time as reported by Google Benchmark.

§ **7.2.2.1 extents<int, 4, 4, 4, 4, 4, 2> (rank 6, IndexType=int, all static extents)**

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	321917	2160
Clang 21.0.0	No	12612291	52
GCC 15.1.0	Yes	322847	2035
GCC 15.1.0	No	322098	2156
Clang 14.0.0	Yes	317226	2160
Clang 14.0.0	No	335550	2151
GCC 11.4.0	Yes	663204	1110
GCC 11.4.0	No	655524	1072

§ **7.2.2.2 dims<6, int>{4, 4, 4, 4, 4, 2} (rank 6, IndexType=int, all dynamic extents)**

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	28573409	25
Clang 21.0.0	No	32210084	22
GCC 15.1.0	Yes	24218781	29
GCC 15.1.0	No	21664452	31
Clang 14.0.0	Yes	16802927	42
Clang 14.0.0	No	13018827	55
GCC 11.4.0	Yes	29826258	25
GCC 11.4.0	No	23318382	30

§ **7.2.2.3 extents<size_t, 4, 4, 4, 4, 4, 2> (rank 6, IndexType=size_t, all static extents)**

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	323624	2099
Clang 21.0.0	No	9560465	22

GCC 15.1.0	Yes	322152	2123
GCC 15.1.0	No	318111	2191
Clang 14.0.0	Yes	326443	2193
Clang 14.0.0	No	9566053	72
GCC 11.4.0	Yes	679657	1044
GCC 11.4.0	No	673962	977

§ 7.2.2.4 `dims<6, size_t>{4, 4, 4, 4, 4, 2}` (rank 6, `IndexType=size_t`, all dynamic extents)

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	11672089	57
Clang 21.0.0	No	29269825	25
GCC 15.1.0	Yes	25481788	27
GCC 15.1.0	No	22775149	32
Clang 14.0.0	Yes	28532969	25
Clang 14.0.0	No	14644212	45
GCC 11.4.0	Yes	21809043	31
GCC 11.4.0	No	26799987	26

§ 7.2.3 Preliminary conclusions

The only conclusion we can make from the above results is that P3663 would not obviously make the reference implementation of `mdspan` slower. Enabling P3663 support improves performance in some cases, especially for Clang 21.

Regardless of whether P3663 support is enabled, compilers vary in their ability to optimize `submdspan` calls. We see this both for different compilers, and for different versions of the same compiler. Optimization tends to be sensitive to code changes, even to not directly related changes like increasing or decreasing the number of lines of code in a file. Understanding why would take a closer analysis of the generated code.

§ 8 Proposed wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

§ 8.1 Increment `__cpp_lib_submdspan` feature test macro

In `[version.syn]`, increase the value of the `__cpp_lib_submdspan` macro by replacing `YYMMML` below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_submdspan YYMMML // also in <mdspan>
```

[Editor's note: If this proposal is adopted for C++26, increasing the version macro is not strictly necessary, because submdspan (P2630) itself is a C++26 feature. We retain the version macro increase in case the proposal is adopted after C++26, possibly as a DR for C++26.]

§ 8.2 Change [mdspan.syn]

Change [mdspan.syn] (“Header <mdspan> synopsis”) as follows.

```
struct full_extent_t { explicit full_extent_t() = default; };
inline constexpr full_extent_t full_extent{};
```

```
template<class IndexType, size_t... Extents, class... Slices>
constexpr auto submdspan_canonicalize_slices(
    const extents<IndexType, Extents...>& src, Slices... slices);
```

```
template<class IndexType, class... Extents, class... SliceSpecifiers>
constexpr auto submdspan_extents(const extents<IndexType,
    Extents...>&, SliceSpecifiers...);
```

§ 8.3 Context from [mdspan.extents.overview] (NO CHANGES)

This proposal does NOT change [mdspan.extents.overview]. We include part of it just for context.

1 *Mandates:*

- (1.1) — IndexType is a signed or unsigned integer type, and
- (1.2) — each element of Extents is either equal to dynamic_extent, or is representable as a value of type IndexType.

2 Each specialization of extents models regular and is trivially copyable.

3 Let E_r be the r^{th} element of Extents. E_r is a *dynamic extent* if it is equal to dynamic_extent, otherwise E_r is a *static extent*. Let D_r be the value of *dynamic-extents[dynamic-index(r)]* if E_r is a dynamic extent, otherwise E_r .

4 The r^{th} interval of the multidimensional index space represented by an extents object is $[0, D_r)$.

§ 8.4 Change [mdspan.sub.overview]

Change [mdspan.sub.overview] as follows.

- 1 The submdspan facilities create a new mdspan viewing a subset of elements of an existing input mdspan. The subset viewed by the created mdspan is determined by the SliceSpecifier arguments.

- 2 For each function defined in [mdspan.sub] that takes a parameter pack named `slices` as an argument:

- (2.1) — let `index_type` be
 - `M::index_type` if the function is a member of a class `M`,
 - otherwise, `remove_reference_t<decltype(src)>::index_type` if the function has a parameter named `src`,
 - otherwise, the same type as the function's template argument `IndexType`;
- (2.2) — let `rank` be the number of elements in `slices`;
- (2.3) — let s_k be the k^{th} element of `slices`;
- (2.4) — let S_k be the type of s_k ; and
- (2.5) — let `map-rank` be an `array<size_t, rank>` such that for each k in the range $[0, \text{rank})$, `map-rank[k]` equals:
 - (2.5.1) — `dynamic_extent` if S_k models `convertible_to<index_type>`,
 - (2.5.2) — otherwise, the number of types S_j with $j < k$ that do not model `convertible_to<index_type>`.

- 3 Given a signed or unsigned integer type `IndexType`, a type S is a *submdspan slice type for `IndexType`* if at least one of the following holds:

- (3.1) — `is_convertible_v<S, full_extent_t>` is true;
- (3.2) — `is_convertible_v<S, IndexType>` is true;
- (3.3) — S is a specialization of `strided_slice` and `is_convertible_v<X, IndexType>` is true for X denoting `S::offset_type`, `S::extent_type` and `S::stride_type`; or
- (3.4) — all of the following hold:
 - (3.4.1) — the declaration `auto [...ls] = std::move(s);` is well-formed for some object `s` of type S ,
 - (3.4.2) — `sizeof...(ls)` is equal to 2, and
 - (3.4.3) — `(is_convertible_v<decltype(std::move(ls))>, IndexType> && ...)` is true.

- 4 Given a signed or unsigned integer type `IndexType`, a type S is a *canonical submdspan index type for `IndexType`* if S is either `IndexType` or `constant_wrapper<v>` for some value `v` of type `IndexType`, such that `v` is greater than or equal to zero.

5 Given a signed or unsigned integer type `IndexType`, a type S is a *canonical submdspan slice type for `IndexType`* if exactly one of the following is true:

(5.1) — S is `full_extent_t`;

(5.2) — S is a canonical submdspan index type for `IndexType`; or

(5.3) — S is a specialization of `strided_slice` where all of the following hold:

(5.3.1) — $S::\text{offset_type}$, $S::\text{extent_type}$, and $S::\text{stride_type}$ are all canonical submdspan index types for `IndexType`; and

(5.3.2) — if $S::\text{stride_type}$ and $S::\text{extent_type}$ are both specializations of `constant_wrapper`, then $S::\text{stride_type}::\text{value}$ is greater than zero.

6 A type S is a *collapsing slice type* if it is neither `full_extent_t` nor a specialization of `strided_slice`.

[*Note 1*: Each collapsing slice type in `submdspan_mapping`'s parameter pack of slice specifier types reduces the rank of the result of `submdspan_mapping` by one. — *end note*]

7 A type S is a *unit-stride slice type* if

(7.1) — S is a specialization of `strided_slice` where $S::\text{stride_type}$ is a specialization of `constant_wrapper` and $S::\text{stride_type}::\text{value}$ is equal to 1, or

(7.1) — S denotes `full_extent_t`.

8 Given an object e of a type E that is a specialization of `extents`, and an object s of type S that is a canonical submdspan slice type for $E::\text{index_type}$, the *submdspan slice range of s for the k^{th} extent of e* is:

(8.1) — $[0, e.\text{extent}(k))$, if S is `full_extent_t`;

(8.2) — $[E::\text{index_type}(s.\text{offset}), E::\text{index_type}(s.\text{offset} + s.\text{extent}))$, if S is a specialization of `strided_slice`; otherwise

(8.3) — $[E::\text{index_type}(s), E::\text{index_type}(s) + 1)$.

9 Given a type E that is a specialization of `extents`, a type S is a *valid submdspan slice type for the k^{th} extent of E* , if S is a canonical slice type for $E::\text{index_type}$, and for x equal to $E::\text{static_extent}(k)$, either x is equal to `dynamic_extent`; or

(9.1) — if S is a specialization of `strided_slice`:

(9.1.1) — if $S::\text{offset_type}$ is a specialization of `constant_wrapper`, then $S::\text{offset_type}::\text{value}$ is less than or equal to x ;

(9.1.2) — if $S::\text{extent_type}$ is a specialization of `constant_wrapper`, then $S::\text{extent_type}::\text{value}$ is less than or equal to x ;

- (9.1.3) — if both `S::offset_type` and `S::extent_type` are specializations of `constant_wrapper`, then `S::offset_type::value + S::extent_type::value` is less than or equal to x ;
- (9.2) — if `S` is a specialization of `constant_wrapper`, then `S::value` is less than x .
- 10 Given an object `e` of type `E` that is a specialization of `extents` and an object `s` of type `S`, `s` is a *valid submdspan slice for the k^{th} extent of `e`* if
 - (10.1) — `S` is a valid submdspan slice type for the k^{th} extent of `E`;
 - (10.2) — the k^{th} interval of `e` contains the submdspan slice range of `s` for the k^{th} extent of `e`; and
 - (10.3) — if `S` is a specialization of `strided_slice`, then:
 - (10.3.1) — `s.extent` is greater than or equal to zero, and
 - (10.3.2) — either `s.extent` equals zero or `s.stride` is greater than zero.

§ 8.5 Change [mdspan.sub.helpers]

Change [mdspan.sub.helpers] as follows.

```
template<class T>
constexpr T de-ice(T val) { return val; }
template<integral-constant-like T>
constexpr auto de-ice(T) { return T::value; }
```

```
template<class IndexType, size_t k, class... SliceSpecifiers>
constexpr IndexType first_(SliceSpecifiers... slices);
```

1 *Mandates:* `IndexType` is a signed or unsigned integer type.

2 Let ϕ_k denote the following value:

- (2.1) — s_k if S_k models `convertible_to<IndexType>`;
- (2.2) — `get<0>(sk)` if S_k models `index-pair-like<IndexType>`;
- (2.3) — `de-ice(sk.offset)` if S_k is a specialization of `strided_slice`;
- (2.4) — otherwise, `0`.

3 *Preconditions:* ϕ_k is representable as a value of type `IndexType`.

4 *Returns:* `extents<IndexType>::index-cast( $\phi_k$ )`.

```
template<size_t k, class Extents, class... SliceSpecifiers>
constexpr auto last_(const Extents& src, SliceSpecifiers... slices);
```

5 *Mandates:* Extents is a specialization of extents.

6 Let `index_type` be `typename Extents::index_type`.

7 Let λ_k denote the following value:

(7.1) — `de-ice( $s_k$ ) + 1` if S_k models `convertible_to<index_type>`; otherwise

(7.2) — `get<1>( $s_k$ )` if S_k models `index-pair-like<index_type>`; otherwise

(7.3) — `de-ice( $s_k$ .offset) + de-ice( $s_k$ .extent)` if S_k is a specialization of `strided_slice`; otherwise

(7.4) — `src.extent(k)`.

8 *Preconditions:* λ_k is representable as a value of type `index_type`.

9 *Returns:* `Extents::index-cast( $\lambda_k$ )`.

```
template<class IndexType, size_t N, class... SliceSpecifiers>
constexpr array<IndexType, sizeof...(SliceSpecifiers)>
src-indices(const array<IndexType, N>& indices, SliceSpecifiers...
            slices);
```

10 *Mandates:* `IndexType` is a signed or unsigned integer type.

11 *Returns:* An `array<IndexType, sizeof...(SliceSpecifiers)> src_idx` such that for each k in the range `[0, sizeof...(SliceSpecifiers))`, `src_idx[k]` equals

(11.1) — `first_<IndexType,  $k$ >(slices...)` for each k where `map-rank[k]` equals `dynamic_extent`,

(11.2) — otherwise, `first_<IndexType,  $k$ >(slices...) + indices[map-rank[k]]`.

1 For a pack `p` and an integer i , let `MAP_RANK(p, i)` be the number of elements `p...[j]` for $0 \leq j < i$ whose types are not collapsing slice types.

```
template<class T>
concept is-strided-slice = see below;
```

2 The concept `is-strided-slice<T>` is satisfied and modeled if and only if `T` is a specialization of `strided_slice`.

```
template<class IndexType, class S>
constexpr auto canonical-index(S s);
```

3 *Mandates:* If `S` models `integral-constant-like`, then `extents<IndexType>::index-cast(S::value)` is representable as a value of type `IndexType`.

4 *Preconditions:* `extents<IndexType>::index-cast(std::move(s))` is representable as a value of type `IndexType`.

5 *Effects*: Equivalent to:

(5.1) — return `cw<IndexType(S::value)>`; if *S* models *integral-constant-like*;

(5.2) — return `IndexType(std::move(s))`; otherwise.

```
template<class IndexType, class S>
constexpr auto canonical-slice(S s);
```

6 *Mandates*: *S* is a submdspan slice type for `IndexType`.

7 *Effects*: Equivalent to:

```
if constexpr (is_convertible_v<S, full_extent_t>)
    return static_cast<full_extent_t>(std::move(s));
else if constexpr (is_convertible_v<S, IndexType>)
    return canonical-index<IndexType>(std::move(s));
else if constexpr (is_strided_slice<S>) {
    auto c_extent = canonical-index<IndexType>(std::move(s.extent));
    auto c_offset = canonical-index<IndexType>(std::move(s.offset));
    if constexpr (is_same_v<decltype(c_extent),
        constant_wrapper<IndexType(0)>>)
        return strided_slice{
            .offset = c_offset,
            .extent = c_extent,
            .stride = cw<IndexType(1)>
        };
    else
        return strided_slice{
            .offset = c_offset,
            .extent = c_extent,
            .stride = canonical-index<IndexType>(std::move(s.stride))
        };
}
else {
    auto [s_first, s_last] = std::move(s);
    auto c_first = canonical-index<IndexType>(std::move(s_first));
    auto c_last = canonical-index<IndexType>(std::move(s_last));
    return strided_slice{
        .offset = c_first,
        .extent = canonical-index<IndexType>(c_last - c_first),
        .stride = cw<IndexType(1)>
    };
}
```

§ 8.6 Add section [mdspan.sub.canonical]

Add a new section [mdspan.sub.canonical] right before [mdspan.sub.extents], with contents as follows.

```
template<class IndexType, size_t... Extents, class... SliceSpecifiers>
constexpr auto submdspan_canonicalize_slices(
    const extents<IndexType, Extents...>& src, SliceSpecifiers... slices);
```

- 1 *Constraints:* `sizeof...(SliceSpecifiers)` equals `sizeof...(Extents)`.
- 2 *Mandates:* For each rank index k of `src`:
 - (2.1) — `SliceSpecifiers...[k]` is a submdspan slice type for `IndexType`, and
 - (2.2) — `decltype(canonical-slice<IndexType>(slices...[k]))` is a valid submdspan slice type for the k^{th} extent of `extents<IndexType, Extents...>`.
- 3 *Preconditions:* For each rank index k of `src`, `canonical-slice<IndexType>(slices...[k])` is a valid submdspan slice for the k^{th} extent of `src`.
- 4 *Returns:* `make_tuple(canonical-slice<IndexType>(slices)...)...`.

§ 8.7 Change section [mdspan.sub.extents]

Change section [mdspan.sub.extents] as follows.

```
template<class IndexType, class... Extents, class... SliceSpecifiers>
constexpr auto submdspan_extents(const extents<IndexType, Extents...>&
    src,
    SliceSpecifiers... raw_slices);
```

- 1 Let `slices` be the pack introduced by the following declaration:


```
auto [...slices] = submdspan_canonicalize_slices(src, raw_slices...);
```
- 2 *Constraints:* ~~`sizeof...(slices)`~~ `sizeof...(SliceSpecifiers)` equals `Extents::rank()`.
- 2 *Mandates:* For each rank index k of `src.extents()`, exactly one of the following is true:
 - (2.1) — S_k models `convertible_to<IndexType>`,
 - (2.2) — S_k models `index-pair-like<IndexType>`,
 - (2.3) — `is_convertible_v< $S_k$ , full_extent_t>` is true, or
 - (2.4) — S_k is a specialization of `strided_slice`.
- 3 *Preconditions:* For each rank index k of `src`, all of the following are true:
 - (3.1) — If S_k is a specialization of `strided_slice`
 - (3.1.1) — $s_k.\text{extent} = 0$, or
 - (3.1.2) — $s_k.\text{stride} > 0$
 - (3.2) — $0 \leq \text{first_}<\text{IndexType}, k>(\text{slices}...) \leq \text{last_}<k>(\text{src}, \text{slices}...) \leq \text{src.extent}(k)$

3 *Mandates:* For each rank index k of `src`:

(3.1) — `SliceSpecifiers...[k]` is a submdspan slice type for `IndexType`, and

(3.2) — `decltype(slices...[k])` is a valid submdspan slice type for the k^{th} extent of `extents<IndexType, Extents...>`.

4 *Preconditions:* For each rank index k of `src`, `slices...[k]` is a valid submdspan slice for the k^{th} extent of `src`.

5 Let `SubExtents` be a specialization of `extents` such that:

(5.1) — `SubExtents::rank()` equals ~~the number of k such that S_k does not model convertible_to<IndexType>~~ `MAP RANK(slices, Extents::rank())`; and

(5.2) — for each rank index k of `Extents` such that ~~`map_rank[k] != dynamic_extent` is true~~ the type of `slices...[k]` is not a collapsing slice type, `SubExtents::static_extent(map_rank[k] MAP RANK(slices, k))` equals the following, where Σ_k denotes the type of `slices...[k]`:

(5.2.1) — `Extents::static_extent(k)` if ~~`is_convertible_v< $S_k$ , full_extent_t>` is true~~ Σ_k denotes full_extent_t; otherwise

(5.2.2) — ~~`de-ice(tuple_element_t<1,  $S_k$ >())`~~ `de-ice(tuple_element_t<0,  $S_k$ >())` if ~~S_k models index-pair-like<IndexType>~~, and both ~~`tuple_element_t<0,  $S_k$ >`~~ and ~~`tuple_element_t<1,  $S_k$ >`~~ model ~~integral-constant-like~~; otherwise

(5.2.3) — 0, if ~~$S_k \Sigma_k$ is a specialization of `strided_slice`, whose `extent_type` models integral-constant-like, for which `extent_type()` equals zero and $\Sigma_k::extent_type$ denotes constant wrapper<IndexType(0)>~~; otherwise

(5.2.4) — ~~`1 + (-de-ice( $S_k::extent\_type()$ ) - 1) / de-ice( $S_k::stride\_type()$ )`~~ `1 + ( $\Sigma_k::extent\_type::value - 1$ ) /  $\Sigma_k::stride\_type::value$` , if ~~$S_k \Sigma_k$ is a specialization of `strided_slice` whose `extent_type` and `stride_type` model integral-constant-like~~ denote specializations of constant wrapper;

(5.2.5) — otherwise, `dynamic_extent`.

6 *Returns:* A value ext of type `SubExtents` such that for each rank index k of `extents<IndexType, Extents...>`, for which ~~`map_rank[k] != dynamic_extent` is true~~ where the type of `slices...[k]` is not a collapsing slice type, `ext.extent(map_rank[k] MAP RANK(slices, k))` equals the following, where σ_k denotes `slices...[k]`:

(6.1) — ~~`$s_k$ .extent == 0 ? 0 : 1 + (de-ice( $s_k$ .extent) - 1) / de-ice( $s_k$ .stride)`~~ `$\sigma_k$ .extent == 0 ? 0 : 1 + ( $\sigma_k$ .extent - 1) /  $\sigma_k$ .stride` if ~~S_k~~ the type of σ_k is a specialization of `strided_slice`,

- (6.2) — otherwise, ~~$\text{last_}<k>(\text{src}, \text{slices} \dots) \rightarrow \text{first_}<\text{IndexType}, k>(\text{slices} \dots)$~~ $U - L$, where $[L, U)$ is the submdspan slice range of σ_k for the k^{th} extent of src .

§ 8.8 Requirements of all submdspan_mapping customizations

§ 8.8.1 New section [mdspan.sub.sliceable], “Sliceable layout mapping requirements”

Right before [mdspan.sub.map.common] (“Specializations of submdspan_mapping”), insert a new section [mdspan.sub.sliceable], “Sliceable layout mapping requirements,” with the following content.

1 Let:

- (1.1) — M denote a layout mapping class;
- (1.2) — IT denote $M::\text{extent_type}::\text{index_type}$;
- (1.3) — m denote a value of type (possibly const) M ;
- (1.4) — M_rank be equal to $M::\text{extent_type}::\text{rank}()$;
- (1.5) — valid_slices denote a pack of (possibly const) objects for which $\text{sizeof} \dots (\text{valid_slices}) == M_rank$ is true and, for each rank index i of $m.\text{extents}()$, $\text{valid_slices} \dots [i]$ is a valid submdspan slice for the i^{th} extent of $m.\text{extents}()$;
- (1.6) — invalid_slices denote a pack of objects for which $\text{sizeof} \dots (\text{invalid_slices}) == M_rank$ is true and there exists an integer k such that the cv-unqualified type of $\text{invalid_slices} \dots [k]$ is none of the following:
 - (1.6.1) — IT ,
 - (1.6.2) — full_extent_t ,
 - (1.6.3) — a specialization of constant_wrapper , or
 - (1.6.3) — a specialization of strided_slice .

2 For the purpose of this section, the meaning of submdspan_mapping is established as if by performing argument-dependent lookup only ([basic.lookup.argdep]).

3 A type M meets the *sliceable layout mapping requirements* if

- (3.1) — M meets the layout mapping requirements ([mdspan.layout.policy.reqmts]),
- (3.2) — the expression $\text{submdspan_mapping}(m, \text{invalid_slices} \dots)$ is ill-formed, and
- (3.3) — the following expression is well-formed and has the specified semantics:

```
submdspan_mapping(m, valid_slices...)
```

4 *Result:* A type SMR that is a specialization of type `submdspan_mapping_result<SM>` for some type SM such that

- (4.1) — SM meets the layout mapping requirements (`[mdspan.layout.policy.reqmts]`),
- (4.2) — `SM::extents_type` is a specialization of `extents`,
- (4.3) — `SM::extents_type::rank()` equals `MAP_RANK(valid_slices, M_rank)`, and
- (4.4) — `SM::extents_type::index_type` denotes IT.

5 *Returns:* An object `smr` of type SMR such that

- (5.1) — `smr.mapping.extents() == submdspan_extents(m.extents(), valid_slices...)` is true; and
- (5.2) — for each integer pack `i` which is a multidimensional index in `smr.mapping.extents()`, `smr.mapping(i...) + smr.offset == m(j)` is true, where `j` is an integer pack such that
 - (5.2.1) — `sizeof...(j)` is equal to `M_rank`; and
 - (5.2.2) — for each rank index `q` of `m.extents()`, `j...[q]` is equal to the sum of
 - (5.2.2.1) — the lower bound of the submdspan slice range of `valid_slices...[q]` for extent `q` of `m.extents()`, and
 - (5.2.2.2) — zero if the type of `valid_slices...[q]` is a collapsing slice type, `i...[MAP_RANK(valid_slices, q)]`, otherwise.

```
template<class LayoutMapping>  
concept sliceable_mapping = see below;
```

6 Let `lm` be an object of type `LayoutMapping` and let `fe` denote a pack of objects of type `full_extent_t` for which `sizeof...(fe) == LayoutMapping::extents_type::rank()` is true. A `LayoutMapping` satisfies *sliceable_mapping* if

- (6.1) — the expression `submdspan_mapping(m, fe...)` is well-formed when treated as an unevaluated operand, and
- (6.2) — the type of that expression is a specialization of `submdspan_mapping_result`.

7 A type `LayoutMapping` models *sliceable_mapping* if `LayoutMapping` meets the sliceable layout mapping requirements.

§ 8.9 Change `[mdspan.sub.map.common]`

Change `[mdspan.sub.map.common]` as follows.

- 1 The following elements apply to all functions in [mdspan.sub.map].
- 2 *Constraints:* `sizeof...(slices)sizeof...(SliceSpecifiers)` equals `extents_type::rank()`.
- 3 *Mandates:* For each rank index k of `extents()`, exactly one of the following is true:
 - (3.1) — S_k models `convertible_to<index_type>`,
 - (3.2) — S_k models `index-pair-like<index_type>`,
 - (3.3) — `is_convertible_v< $S_k$ , full_extent_t>` is true, or
 - (3.4) — S_k is a specialization of `strided_slice`.
- 4 *Preconditions:* For each rank index k of `extents()`, all of the following are true:
 - (4.1) — if S_k is a specialization of `strided_slice`, s_k .`extent` is equal to zero or s_k .`stride` is greater than zero; and
 - (4.2) — $0 \leq \text{first_}<\text{index_type}, k>(\text{slices} \dots)$
 $0 \leq \text{last_}<k>(\text{extents}(), \text{slices} \dots)$
 $0 \leq \text{extents}().\text{extent}(k)$
- 3 *Mandates:* For each rank index k of `extents()`, `SliceSpecifiers...[k]` is a valid submdspan slice type for the k^{th} extent of `Extents`.
- 4 *Preconditions:* For each rank index k of `extents()`, `slices...[k]` is a valid slice for the k^{th} extent of `extents()`.
- 5 Let `sub_ext` be the result of `submdspan_extents(extents(), slices...)` and let `SubExtents` be `decltype(sub_ext)`.
- 6 Let `sub_strides` be an `array<SubExtents::index_type, SubExtents::rank()>` such that for each rank index k of `extents()` for which ~~`map=rank[k]` is not `dynamic_extent`~~ the type of `slices...[k]` is not a collapsing slice type, `sub_strides[map=rank[k]MAP RANK(slices,k)]` equals:
 - (6.1) — `stride(k) *de-ice(sk, stride)s.stride` if S_k is a specialization of `strided_slice` and ~~s_k .`stride` < s_k .`extent`~~ `s.stride` < `s.extent` is true where `s` is `slices...[k]`;
 - (6.2) — otherwise, `stride(k)`.
- 7 ~~Let `P` be a parameter pack such that `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>` is true.~~ Let `ls` be a pack of values of `index_type`, where the q^{th} element equals the lower bound of the submdspan slice range of `slices...[q]` for extent q of `extents()`.

8 If ~~`first_index_type, k`~~(~~`slices...`~~)`ls...``[k]` equals `extents().extent(k)` for any rank index `k` of `extents()`, then let `offset` be a value of type `size_t` equal to ~~`(*this)`~~`required_span_size()`. Otherwise, let `offset` be a value of type `size_t` equal to ~~`(*this)`~~`operator()`( ~~`first_index_type, P`~~(~~`slices...`~~)~~`...`~~`ls...`).

9 Given a layout mapping type `M`, a type `S` is a *unit-stride slice for M* if

- (9.1) — `S` is a specialization of `strided_slice` where `S::stride_type` models *integral-constant-like* and `S::stride_type::value` equals 1,
- (9.2) — `S` models *index-pair-like*`<M::index_type>`, or
- (9.3) — `is_convertible_v<S, full_extent_t>` is true.

[Editor's note: The definition of unit-stride slice has been moved up to [\[mdspan.sub.overview\]](#)]

§ 8.10 Change `[mdspan.sub.map.left]`, `layout_left` specialization of `submdspan_mapping`

Change `[mdspan.sub.map.left]` (“`layout_left` specialization of `submdspan_mapping`”) as follows.

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_left::mapping<Extents>::submdspan_mapping_impl(
    SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank() == 0` is true;
- (1.2) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if `SubExtents::rank() == 0` is true;
- (1.3) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if
 - (1.3.1) — for each `k` in the range `[0, SubExtents::rank() - 1)`, ~~`is_convertible_v<Sk,`~~
~~`full_extent_t> is true`~~`SliceSpecifiers...[k]` denotes *full extent t*; and
 - (1.3.2) — for `k` equal to `SubExtents::rank() - 1`, ~~`Sk`~~`SliceSpecifiers...[k]` is a unit-stride slice *type for mapping*;

[Note: If the above conditions are true, all ~~`Sk`~~`SliceSpecifiers...[k]` with `k` larger than `SubExtents::rank() - 1` are convertible to `index_type`. - end note]

- (1.4) — otherwise, `submdspan_mapping_result{layout_left_padded<S_static>::mapping(sub_ext, stride(u+1)), offset}`, if for a value `u` for which `u + 1` is the smallest value `p`

larger than zero for which $S_p\text{SliceSpecifiers...}[p]$ is a unit-stride slice type for mapping, the following conditions are met:

- (1.4.1) — $S_0\text{SliceSpecifiers...}[0]$ is a unit-stride slice type for mapping; and
- (1.4.2) — for each k in the range $[u + 1, u + \text{SubExtents}::\text{rank}() - 1]$, ~~$\text{is_convertible_v} \langle S_k, \text{full_extent_t} \rangle \text{ is true}$~~ $\text{SliceSpecifiers...}[k]$ denotes full extent t ; and
- (1.4.3) — for k equal to $u + \text{SubExtents}::\text{rank}() - 1$, $S_k\text{SliceSpecifiers...}[k]$ is a unit-stride slice type for mapping;

and where S_static is:

- (1.4.4) — dynamic_extent , if $\text{static_extent}(k)$ is dynamic_extent for any k in the range $[0, u + 1]$;
- (1.4.5) — otherwise, the product of all values $\text{static_extent}(k)$ for k in the range $[0, u + 1]$;
- (1.5) — otherwise, $\text{submdspan_mapping_result}\{\text{layout_stride}::\text{mapping}(\text{sub_ext}, \text{sub_strides}), \text{offset}\}$

§ 8.11 Change [mdspan.sub.map.right], layout_right specialization of submdspan_mapping

Change [mdspan.sub.map.right] (“layout_right specialization of submdspan_mapping”) as follows.

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_right::mapping<Extents>::submdspan_mapping_impl(
    SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

- (1.1) — $\text{submdspan_mapping_result}\{*this, 0\}$, if $\text{Extents}::\text{rank}() == 0$ is true;
- (1.2) — otherwise, $\text{submdspan_mapping_result}\{\text{layout_right}::\text{mapping}(\text{sub_ext}), \text{offset}\}$, if $\text{SubExtents}::\text{rank}() == 0$ is true;
- (1.3) — otherwise, $\text{submdspan_mapping_result}\{\text{layout_left_right}::\text{mapping}(\text{sub_ext}), \text{offset}\}$, if

[Editor's note: Please note drive-by fix in 1.3.]

- (1.3.1) — for each k in the range $[\text{rank_} - \text{SubExtents}::\text{rank}() + 1, \text{rank_}]$, ~~$\text{is_convertible_v} \langle S_k, \text{full_extent_t} \rangle \text{ is true}$~~ $\text{SliceSpecifiers...}[k]$ denotes full extent t ; and

- (1.3.2) — for k equal to $\text{rank_rank} - \text{SubExtents}::\text{rank}()$, $S_k\text{SliceSpecifiers}...[k]$ is a unit-stride slice **type for mapping**;

[Editor's note: Please note drive-by fix in 1.3.2.]

[Note: If the above conditions are true, all $S_k\text{SliceSpecifiers}...[k]$ with $k < \text{rank_rank} - \text{SubExtents}::\text{rank}()$ are convertible to `index_type`. - end note]

- (1.4) — otherwise,
- submdspan_mapping_result{layout_right_padded<S_static>::mapping(sub_ext, stride(rank_ - u - 2)), offset} if for a value u for which $\text{rank_rank} - u - 2$ is the largest value p smaller than $\text{rank_rank} - 1$ for which $S_p\text{SliceSpecifiers}...[p]$ is a unit-stride slice **type for mapping**, the following conditions are met:
- (1.4.1) — for k equal to $\text{rank_rank} - 1$, $S_k\text{SliceSpecifiers}...[k]$ is a unit-stride slice **type for mapping**; and
- (1.4.2) — for each k in the range $[\text{rank_rank} - \text{SubExtents}::\text{rank}() - u + 1, \text{rank_rank} - u - 1]$, ~~$\text{is_convertible_v}<S_k, \text{full_extent_t}>\text{is_true}$~~ $S_k\text{SliceSpecifiers}...[k]$ **denotes full extent t**; and
- (1.4.3) — for k equal to $\text{rank_rank} - \text{SubExtents}::\text{rank}() - u$, $S_k\text{SliceSpecifiers}...[k]$ is a unit-stride slice **type for mapping**;

and where `S_static` is:

- (1.4.4) — `dynamic_extent`, if `static_extent(k)` is `dynamic_extent` for any k in the range $[\text{rank_rank} - u - 1, \text{rank_rank}]$,
- (1.4.5) — otherwise, the product of all values `static_extent(k)` for k in the range $[\text{rank_rank} - u - 1, \text{rank_rank}]$;
- (1.5) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

§ 8.12 Do not change `[mdspan.sub.map.stride]`, `layout_stride` specialization of `submdspan_mapping`

There are no changes to `[mdspan.sub.map.stride]` (“`layout_stride` specialization of `submdspan_mapping`”). We retain the full section here for context.

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_stride::mapping<Extents>::submdspan_mapping_impl(
    SliceSpecifiers ... slices) const -> see below;
```

1 *Returns:*

- (1.1) — submdspan_mapping_result{*this, 0}, if Extents::rank() == 0 is true;
- (1.2) — otherwise, submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}

§ 8.13 Change [mdspan.sub.map.leftpad], layout_left_padded specialization of submdspan_mapping

Change [mdspan.sub.map.leftpad] (“layout_left_padded specialization of submdspan_mapping”) as follows.

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_left_padded::mapping<Extents>::submdspan_mapping-
impl(
    SliceSpecifiers... slices) const -> see below;
```

1 Returns:

- (1.1) — submdspan_mapping_result{*this, 0}, if Extents::rank() == 0 is true;
- (1.2) — otherwise, submdspan_mapping_result{layout_left::mapping(sub_ext), offset}, if rank_ == 1 is true or SubExtents::rank() == 0 is true;
- (1.3) — otherwise, submdspan_mapping_result{layout_left::mapping(sub_ext), offset}, if
 - (1.3.1) — SubExtents::rank() == 1 is true and
 - (1.3.2) — S_0 SliceSpecifiers...[0] is a unit-stride slice type for mapping;
 - (1.4) — otherwise,

submdspan_mapping_result{layout_left_padded<S_static>::mapping(sub_ext, stride(u + 1)), offset}, if for a value u for which $u + 1$ is the smallest value p larger than zero for which S_p SliceSpecifiers...[p] is a unit-stride slice type for mapping, the following conditions are met:

 - (1.4.1) — S_0 SliceSpecifiers...[0] is a unit-stride slice type for mapping; and
 - (1.4.2) — for each k in the range $[u + 1, u + \text{SubExtents::rank()} - 1)$, ~~is_convertible_v< S_k , full_extent_t> is true~~SliceSpecifiers...[k] denotes full extent t; and
 - (1.4.3) — for k equal to $u + \text{SubExtents::rank()} - 1$, S_k SliceSpecifiers...[k] is a unit-stride slice type for mapping;

where S_static is:

- (1.4.4) — `dynamic_extent`, if `static-padding-stride` is `dynamic_extent` or `static_extent(k)` is `dynamic_extent` for any k in the range $[1, u + 1)$,
- (1.4.5) — otherwise, the product of `static-padding-stride` and all values `static_extent(k)` for k in the range $[1, u + 1)$;
- (1.5) — otherwise, `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

§ 8.14 Change `[mdspan.sub.map.rghtpad]`, `layout_right_padded` specialization of `submdspan_mapping`

Change `[mdspan.sub.map.rghtpad]` (“`layout_right_padded` specialization of `submdspan_mapping`”) as follows.

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_right_padded::mapping<Extents>::submdspan_mapping-
impl(
    SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `rank_ == 0` is true;
- (1.2) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if `rank_ == 1` is true or `SubExtents::rank() == 0` is true;
- (1.3) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if
 - (1.3.1) — `SubExtents::rank() == 1` is true and
 - (1.3.2) — for k equal to `rank_ - 1`, `$S_k$ SliceSpecifiers...[k]` is a unit-stride slice `type for mapping`;
- (1.4) — otherwise,
 `submdspan_mapping_result{layout_right_padded<S_static>::mapping(sub_ext, stride(rank_ - u - 2)), offset}` if for a value u for which `rank_ - u - 2` is the largest value p smaller than `rank_ - 1` for which `$S_p$ SliceSpecifiers...[p]` is a unit-stride slice `type for mapping`, the following conditions are met:
 - (1.4.1) — for k equal to `rank_ - 1`, `$S_k$ SliceSpecifiers...[k]` is a unit-stride slice `type for mapping`; and
 - (1.4.2) — for each k in the range $[rank_ - SubExtents::rank() - u + 1, rank_ - u - 1)$, `is_convertible_v< $S_k$ , full_extent_t>is trueSliceSpecifiers...[k]`

denotes full extent t ; and [Editor's note: Please note drive-by fix (removal of close parenthesis)]

- (1.4.3) — for k equal to $\text{rank_} - \text{SubExtents::rank}() - u$, $S_k\text{SliceSpecifiers...}[k]$ is a unit-stride slice type for mapping;

and where S_static is:

- (1.4.4) — dynamic_extent if $\text{static-padding-stride}$ is dynamic_extent or for any k in the range $[\text{rank_} - u - 1, \text{rank_} - 1)$ $\text{static_extent}(k)$ is dynamic_extent ,

- (1.4.1) — otherwise, the product of $\text{static-padding-stride}$ and all values $\text{static_extent}(k)$ with k in the range $[\text{rank_} - u - 1, \text{rank_} - 1)$;

- (1.5) — otherwise, $\text{submdspan_mapping_result}\{\text{layout_stride}::\text{mapping}(\text{sub_ext}, \text{sub_strides}), \text{offset}\}$

§ 8.15 Change [mdspan.sub.sub], submdspan function template

Change [mdspan.sub.sub] (“submdspan function template”) as follows.

```
template<class ElementType, class Extents, class LayoutPolicy,
        class AccessorPolicy, class... SliceSpecifiers>
constexpr auto submdspan(
    const mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>& src,
    SliceSpecifiers... raw_slices) -> see below;
```

- 1 Let index_type be `typename Extents::index_type`.

- 2 Let slices be the pack introduced by the following declaration:

```
auto [...slices] = submdspan_canonicalize_slices(src, raw_slices...);
```

- 3 Let sub_map_offset be the result of $\text{submdspan_mapping}(\text{src.mapping}(), \text{slices...})$.

[Note 1: This invocation of submdspan_mapping selects a function call via overload resolution on a candidate set that includes the lookup set found by argument-dependent lookup ([contents]). — end note]

- 4 *Constraints:*

- (4.1) — $\text{sizeof...}(\text{slices})$ equals $\text{Extents::rank}()$, and

- (4.2) — ~~the expression $\text{submdspan_mapping}(\text{src.mapping}(), \text{slices...})$ is well-formed when treated as an unevaluated operand~~ $\text{LayoutPolicy::mapping<Extents>}$ models sliceable_mapping .

- 5 *Mandates:* For each rank index k of src :

- (5.1) — $\text{SliceSpecifiers...}[k]$ is a submdspan slice type for index_type , and

- (5.2) — `decltype(slices...[k])` is a valid submdspan slice type for the k^{th} extent of Extents.
- (5.1) — `decltype(submdspan_mapping(src.mapping(), slices...))` is a specialization of `submdspan_mapping_result`.
- (5.2) — `is_same_v<remove_cvref_t<decltype(sub_map_offset.mapping.extents())>, decltype(submdspan_extents(src.mapping(), slices...))>` is true.
- (5.3) — For each rank index k of `src.extents()`, exactly one of the following is true:
 - (5.3.1) — S_k models `convertible_to<index_type>`,
 - (5.3.2) — S_k models `index-pair-like<index_type>`,
 - (5.3.3) — `is_convertible_v< $S_k$ , full_extent_t>` is true, or
 - (5.3.4) — S_k is a specialization of `strided_slice`.

6 *Preconditions:* For each rank index k of `src.extents()`, `slices...[k]` is a valid submdspan slice for the k^{th} extent of `src.extents()`.

- (6.1) — For each rank index k of `src.extents()`, all of the following are true:
 - (6.1.1) — if S_k is a specialization of `strided_slice`
 - (6.1.1.1) — `sk.extent = 0`, or
 - (6.1.1.2) — `sk.stride > 0`
 - (6.1.2) — `0 ≤ first_<index_type, k>(slices) ≤ last_<k>(src.extents(), slices...) ≤ src.extent(k)`
- (6.2) — `sub_map_offset.mapping.extents() == submdspan_extents(src.mapping(), slices...)` is true; and
- (6.3) — for each integer pack `I` which is a multidimensional index in `sub_map_offset.mapping.extents()`,

```
sub_map_offset.mapping(I...) + sub_map_offset.offset ==
src.mapping()(src-indices(array{I...}, slices...))
```

is true.

[Note 2: These conditions ensure that the mapping returned by `submdspan_mapping` matches the algorithmically expected index-mapping given the slice specifiers. — *end note*]

7 *Effects:* Equivalent to:

```
auto sub_map_result =  
    submdspan_mapping(src.mapping(), slices...);  
return mdspan(  
    src.accessor().offset(src.data_handle(), sub_map_result.offset),  
    sub_map_result.mapping,  
    typename AccessorPolicy::offset_policy(src.accessor()));
```

[Editor's note: Please note drive-by fix for `src.data_handle()` (original was `src.data()`).]

[Editor's note: Please note drive-by fix adding missing typename before `AccessorPolicy::offset_policy`.]